Package 'rfsnEfficiency'

July 2, 2026

Title Risk-Reduction Efficacy and Efficiency of Risk-Management Configurations

Version 0.0.0.9000

Description A general, on-demand toolkit for evaluating risk-management or safety-net configurations. From user-supplied end-of-season outcome draws (ideally one element per agent) and program parameters (cost, premium, subsidy, and an indemnification rule that may be a function), it builds a configurable outcome (revenue, profit, margin, or a custom / cost-side measure) relative to a no-program baseline and computes efficacy scores (mean lift, variability reduction) and a risk-reduction efficiency ratio, with comparisons across regimes or scenarios. Implements the risk-reduction-efficiency metric of Tsiboe et al. (2025).

License GPL (>= 3)

Encoding UTF-8

LazyData true

Imports arpcFCIPcalc,
data.table,
stats,
utils

Suggests testthat (>= 3.0.0),
piggyback,
usethis,
knitr,
rmarkdown

VignetteBuilder knitr

Config/testthat/edition 3

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Depends R (>= 3.5)

Contents

`assemble_scenario_outcomes` *Assemble scenario outcome columns from a baseline plus a component recipe*

Description

Builds one outcome column per scenario as `baseline + sum(scale * component)`, driven by a long recipe table. Each recipe row adds one component column (optionally scaled by another column, e.g. an adoption rate) onto the named scenario; scenarios with no component rows (or a single `NA` component) equal the baseline. Analysis-agnostic: the scenario design lives entirely in `recipe`.

Usage

```
assemble_scenario_outcomes(data, baseline, recipe)
```

Arguments

`data`	data.table containing `baseline` and every `component`/`scale` column referenced by `recipe`.
`baseline`	name of the baseline outcome column added to every scenario.
`recipe`	data.table with columns `scenario` (output column name), `component` (column to add; `NA` = baseline only), and `scale` (optional column to multiply the component by; `NA` = unscaled).

Value

`data` (modified in place) with one new column per unique `scenario`.

`build_agent_outcomes` *Build per-agent baseline/treated outcomes from end-of-season draws + parameters*

Description

The flexible entry point: hand in each agent's underlying end-of-season outcomes and the program parameters, and get back stacked baseline/treated outcome draws ready for `compute_efficiency_scores()`.

Usage

```
build_agent_outcomes(
  agents,
  indemnity,
  outcome = "outcome",
  cost = 0,
  premium = 0,
  subsidy = 0,
  agent_id_col = "agent_id",
  extra_cols = NULL,
  floor_at = 0
)
```

Arguments

`agents`	a list with one element per agent. Each element is either a numeric vector of the base end-of-season outcome (draws), or a data.frame/data.table of draws (rows = crop years / Monte Carlo draws) containing at least the `outcome` column. List names, if present, become agent ids.
`indemnity`	indemnification rule: a `function(agent_df)` returning a per-draw indemnity vector, OR a column name in each agent's data, OR a numeric scalar/vector. Indemnity is ADDED to the treated outcome.
`outcome`	name of the base end-of-season outcome column (default "outcome"); ignored when an agent element is a plain numeric vector.
`cost`, `premium`, `subsidy`	program parameters, each a `function(agent_df)`, a column name, or a numeric scalar/vector. `cost` is subtracted from the base outcome (enters both baseline and treated). `premium` is the producer-paid premium, subtracted from the treated outcome. `subsidy` is carried through for program-context summaries (not added to the outcome).
`agent_id_col`	name of the agent id column in the output (default "agent_id").
`extra_cols`	optional character vector of additional per-draw columns to carry through from each agent's data (e.g., "commodity_code", "combination", "regime", "commodity_year").
`floor_at`	numeric floor applied to both outcome series (default 0; `NA` = none).

Details

$$outcome\_baseline = outcome - cost$$
$$outcome\_treated = outcome\_baseline + indemnity - producer\_premium$$

Value

a data.table stacked over agents with agent_id, draw, outcome_baseline, outcome_treated, and the resolved cost, producer_premium, subsidy, indemnity columns (plus extra_cols).

Examples

```
# Two agents, each with 100 end-of-season revenue draws; indemnity is a
# function paying 70% of the shortfall below a 600 guarantee.
set.seed(1)
agents <- list(
  cornA = data.frame(outcome = pmax(0, rnorm(100, 600, 150))),
  cornB = data.frame(outcome = pmax(0, rnorm(100, 550, 200)))
)
indem <- function(df) pmax(0, 600 - df$outcome) * 0.70
d <- build_agent_outcomes(agents, indemnity = indem, premium = 25, subsidy = 0.62)
compute_efficiency_scores(d, by = "agent_id")[, .(agent_id, mean_index, risk_index, efficiency)]
```

---

build_outcome *Construct the analysis outcome and its no-safety-net baseline*

Description

Construct the analysis outcome and its no-safety-net baseline

Usage

```
build_outcome(
  data,
  base_value = "revenue",
  cost = NULL,
  transfers = "indemnity",
  cost_protection = NULL,
  premium = "producer_premium",
  floor_at = 0
)
```

Arguments

data	data.frame/data.table with the component columns.
base_value	column name of the gross pre-safety-net outcome (e.g., "revenue"; for profit pass revenue plus a cost).
cost	optional column subtracted from base_value. May be a stochastic per-observation series so cost risk enters the outcome (enabling cost-side products). NULL = revenue; a production-cost column = profit.
transfers	character vector of revenue/yield-side payout columns ADDED to the treated outcome (e.g., c("indemnity","title1_payment")). NULL for products that provide no revenue-side protection.
cost_protection	character vector of cost-side payout columns (a product that pays when costs rise) ADDED to the treated outcome. NULL if none.

premium — column SUBTRACTED from the treated outcome (producer-paid premium for whichever product(s) are active); NULL to ignore.

floor_at — numeric floor applied to both series (default 0; NA = none).

Value

data with new columns outcome_baseline and outcome_treated.

Examples

```
## Not run:
build_outcome(d, base_value = "revenue", cost = NULL)                # revenue
build_outcome(d, base_value = "revenue", cost = "input_cost")        # profit
build_outcome(d, "revenue", cost = "input_cost", transfers = NULL,    # cost-only
              cost_protection = "margin_indemnity", premium = "margin_premium")

## End(Not run)
```

---

compute_efficiency_scores

Compute efficacy and efficiency scores from a generic outcome

---

Description

Compute efficacy and efficiency scores from a generic outcome

Usage

```
compute_efficiency_scores(
  data,
  by,
  outcome_baseline = "outcome_baseline",
  outcome_treated = "outcome_treated"
)
```

Arguments

data — data.frame/data.table with grouping columns and the two outcome series (outcome_baseline, outcome_treated), one row per draw/year within each group.

by — character vector of grouping columns (the unit the scores describe, e.g., agent identifiers x combination).

outcome_baseline, outcome_treated — column names of the no-safety-net and with-safety-net outcome series.

Value

data.table, one row per by group, with moments (*_base/*_sn), relative indices (*_index), headline scores (mean_index, risk_index, efficiency), flags, percent transforms, and the downside/shortfall flavors.

efficiency_score_columns

Canonical column names emitted by compute_efficiency_scores()

Description

Returns the score / indicator column names produced by `compute_efficiency_scores`, grouped by how they should be summarised: continuous `scores` (indices, efficiency, percent transforms), the baseline/safety-net `moments`, and the logical `flags`. Used as the default column set by `summarise_scores` and available for callers that want to summarise or difference the same set.

Usage

```
efficiency_score_columns()
```

Value

named list with character vectors `scores`, `moments`, and `flags`.

efficiency_score_crosswalk

Crosswalk: intuitive names <-> legacy / source-pipeline column names

Description

Maps the package's intuitive score names to the terse column names used by earlier implementations of the same metrics (e.g., `its`, `Relcv`, `sner1`).

Usage

```
efficiency_score_crosswalk()
```

Value

data.frame(`score`, `legacy`, `meaning`).

explode_draws *Explode nested draw list-columns to long form*

Description

Unnests one or more equal-length list-columns (e.g. per-agent Monte-Carlo draw vectors) into a long table with one row per input row x draw element. Every column that is neither a draw column nor dropped is carried (recycled) onto the expanded rows. Analysis-agnostic.

Usage

```
explode_draws(
  data,
  draw_columns,
  value_names = draw_columns,
  drop = character(0)
)
```

Arguments

data	data.frame/data.table with the list-columns named in draw_columns; all list-columns must have the same length within a row.
draw_columns	character vector of list-column names to unnest.
value_names	names for the unnested output columns (defaults to draw_columns).
drop	character vector of columns to discard (e.g. draw metadata not needed downstream).

Value

a data.table, long over draws.

income_transfer_score *Income Transfer Score (ITS)*

Description

Ratio of the expected outcome with the safety net to the expected outcome without it. ITS > 1 indicates the configuration raises the mean outcome. (Eq. 2 in Tsiboe et al. 2025.)

Usage

```
income_transfer_score(treated_mean, baseline_mean)
```

Arguments

treated_mean, baseline_mean	numeric vectors of mean outcomes with and without the safety net.

Value

numeric vector (non-finite ratios returned as NA).

regime_deltas *Regime deltas (alt - ref) for every numeric score*

Description

Regime deltas (alt - ref) for every numeric score

Usage

```
regime_deltas(
  scores,
  by,
  metrics = NULL,
  regime_col = "regime",
  ref = NULL,
  alt = NULL
)
```

Arguments

scores	data.table from compute_efficiency_scores() stacked over regimes (must include regime_col).
by	grouping columns identifying a comparable unit across regimes.
metrics	numeric score columns to difference (default: all numeric).
regime_col	regime column name (default "regime").
ref, alt	reference and alternative regime labels. If NULL (default), the first two (sorted) regime values are used.

Value

data.table with by, metric, the two regime columns, and delta.

risk_reduction_efficiency *Risk Reduction Efficiency Ratio (RRER) – Equation (3)*

Description

$$RRER = \mathbb{1}(VRS < 1)\mathbb{1}(ITS > 1)\,\frac{1 - VRS}{ITS - 1}$$

Usage

```
risk_reduction_efficiency(its, vrs)
```

Arguments

its	Income Transfer Score (from income_transfer_score()).
vrs	Variability Reduction Score (from variability_reduction_score()).

Details

The proportional variability reduction deflated by the proportional income (outcome) gain required to achieve it. It is `0` unless the configuration both lifts the mean (`ITS > 1`) and cuts variability (`VRS < 1`); otherwise it ranges over `[0, Inf)`, with larger values = more efficient risk mitigation. Exactly Equation (3) of Tsiboe et al. (2025).

Value

numeric vector in `[0, Inf)`.

Examples

```
risk_reduction_efficiency(its = 1.05, vrs = 0.80)   # both conditions met
risk_reduction_efficiency(its = 0.98, vrs = 0.80)   # mean not lifted -> 0
```

---

run_efficiency_analysis

Run an on-demand efficiency analysis

Description

High-level driver: given simulated outcomes (one row per draw/year x group, optionally tagged by regime/scenario), build the outcome if needed, compute the efficacy/efficiency scores per regime, and difference regimes.

Usage

```
run_efficiency_analysis(
  data,
  by,
  regime_col = "regime",
  outcome_args = NULL,
  ref = NULL,
  alt = NULL
)
```

Arguments

`data`	data.frame/data.table of simulated outcomes. Must contain the by columns and either (`outcome_baseline`, `outcome_treated`) or the raw components that `build_outcome()` needs (then pass `outcome_args`).
`by`	character vector of grouping columns (e.g., `c("group_id","scenario")`).
`regime_col`	regime column name (default "regime"); created as "all" if absent.
`outcome_args`	named list of arguments forwarded to `build_outcome()` when the outcome columns are not already present (e.g., `list(base_value = "outcome", transfers = "indemnity", premium = "premium")`).
`ref, alt`	reference and alternative regime labels for the deltas. If `NULL` (default) and exactly/at least two regimes are present, the first two (sorted) are used.

Value

list with `scores` (per regime) and `deltas` (alt - ref).

Examples

```
# Self-contained: synthetic outcomes, no external data required.
d <- simulate_example_outcomes()
res <- run_efficiency_analysis(
  data = d, by = c("group_id", "scenario"),
  outcome_args = list(base_value = "outcome", transfers = "indemnity",
                      premium = "premium"))
head(res$scores)
head(res$deltas)
```

`score_dictionary` *Score and indicator dictionary*

Description

A committed look-up table explaining every column produced by `compute_efficiency_scores()` – the moments, relative indices, headline scores, efficiency ratios, flags, and percent transforms – plus the naming conventions used by `summarise_scores()` (`n_producers`, `med_*`, `pct_*`) and the report-pipeline `d_*` deltas. Use it to look up what a variable means, how it is calculated, and how to read it. Metrics follow Tsiboe et al. (2025).

Usage

```
score_dictionary
```

Format

A `data.frame` with one row per variable (or naming convention) and columns:

variable the column name (or `<...>` naming pattern) it documents.

group family: `moment`, `index`, `score`, `flag`, `percent`, `summary`, or `delta`.

description what the quantity is.

calculation how it is computed (in terms of the moments/indices).

interpretation how to read its value.

Source

Built internally

score_scenarios *Score many outcome columns against a common baseline*

Description

Loops over `outcome_cols`, scoring each against `baseline_col` with `compute_efficiency_scores()` (draws = the rows within each by group), and row-binds the per-scenario scores. Optionally merges a `scenario_map` of labels and attaches a per-group `weight`. Analysis-agnostic wrapper.

Usage

```
score_scenarios(
  data,
  outcome_cols,
  by,
  baseline_col = "revenue00",
  scenario_map = NULL,
  weight = NULL,
  scenario_col = "scenario_col"
)
```

Arguments

`data`	data.table with by, `baseline_col`, and every `outcome_cols` column; one row per group x draw.
`outcome_cols`	character vector of treated outcome columns to score.
`by`	character vector of grouping columns (the scored unit; draws are the rows within each group).
`baseline_col`	name of the no-treatment baseline column.
`scenario_map`	optional data.table keyed by `scenario_col` to merge scenario labels onto the scores.
`weight`	optional name of a per-group weight column in `data` to carry onto the scores.
`scenario_col`	name of the column identifying the scored outcome column (default `"scenario_col"`).

Value

a data.table of scores, one row per by group x scenario.

simulate_basic_policy_net_benefit *Simulate a basic (individual) policy's per-draw net benefit*

Description

Adds an individual, loss-contingent indemnity and a scenario net benefit to a draw-level agent table. The indemnity is the M-13 payment rate applied to the simulated farm yield draw (`yield_farm`) against the guarantee, times the underlying liability; it is triggered at the coverage level (full payout at total loss). Because the indemnity does not depend on the subsidy scenario it is computed once (guarded on `underlying_indemnity_amount`) and reused.

Usage

```
simulate_basic_policy_net_benefit(
  data,
  subsidy_amount,
  net_benefit_label,
  indemnity_label = NULL
)
```

Arguments

data	data.table exploded to one row per agent x draw. Must contain underlying_liability_amount, underlying_total_premium_amount, approved_yield, yield_farm, harvest_price, projected_price, coverage_level_percent, insurance_plan_code, and the subsidy_amount column named below.
subsidy_amount	name of the per-draw subsidy-amount column to add in.
net_benefit_label	name of the net-benefit column to create (indemnity - premium + subsidy).
indemnity_label	optional name for a copy of the per-draw indemnity to keep on data. NULL (default) leaves only underlying_indemnity_amount.

Value

data (modified in place); invisibly, with underlying_indemnity_amount, net_benefit_label, and (if requested) indemnity_label added.

---

simulate_example_outcomes

Simulate a tiny example outcomes dataset (no external data required)

Description

Generates synthetic per-draw outcomes for two regimes and a few scenarios so the package can be demonstrated, tested, and learned standalone. This is an ILLUSTRATIVE generator, not a calibrated model – the numbers carry no meaning.

Usage

```
simulate_example_outcomes(
  n_groups = 6,
  n_draws = 200,
  regimes = c("baseline", "reform"),
  seed = 1
)
```

Arguments

n_groups	number of groups (e.g., units / commodities / portfolios).
n_draws	draws (years / Monte Carlo) per group per scenario per regime.
regimes	character vector of regime labels.
seed	RNG seed for reproducibility.

Value

a data.table with group_id, scenario, regime, outcome, indemnity, and premium.

Examples

```
d <- simulate_example_outcomes(n_groups = 3, n_draws = 50)
head(d)
```

---

simulate_supplemental_net_benefit

Simulate a supplemental (area) SCO/ECO endorsement's per-draw net benefit

---

Description

Adds an area-triggered, loss-contingent supplemental net benefit to a draw-level agent table. Supplemental liability is the underlying liability scaled to the endorsement band (area_loss_start_percent - area_loss_end_percent); premium and subsidy are actuarial. The indemnity is the M-13 payment rate on the COUNTY (pool) draw (final_county_yield vs expected_county_yield), so payouts follow the area outcome and basis risk is preserved. trigger_index is the band top (coverage ratio) and coverage_range the band width; the underlying insurance_plan_code selects yield vs revenue-to-count logic.

Usage

```
simulate_supplemental_net_benefit(
  data,
  area_loss_start_percent,
  area_loss_end_percent,
  base_rate,
  rate_differential = NULL,
  subsidy_percent,
  net_benefit_label,
  indemnity_label = NULL
)
```

Arguments

data	data.table exploded to one row per agent x draw. Must contain underlying_liability_amount, coverage_level_percent, expected_county_yield, final_county_yield, harvest_price, projected_price, insurance_plan_code, and the band/rate/subsidy columns named by the string arguments below.
area_loss_start_percent, area_loss_end_percent	column names of the band top and bottom (coverage ratios).
base_rate	column name of the supplemental base premium rate.
rate_differential	optional column name of a multiplicative premium-rate differential (OBBBA recalibrations); NULL uses base_rate alone.
subsidy_percent	column name of the supplemental subsidy share.

`net_benefit_label`
: name of the net-benefit column to create (`indemnity - premium + subsidy`).

`indemnity_label`
: optional name under which to keep a copy of the per-draw supplemental indemnity. `NULL` (default) drops it with the other intermediates.

Value

`data` (modified in place); the intermediate supplemental columns are dropped, leaving `net_benefit_label` (and `indemnity_label` if requested).

`summarise_scores` *Summarise producer-level scores into robust group summaries*

Description

Collapses the per-producer score table from `compute_efficiency_scores()` to one row per `by` group. By default it summarises **every** score, indicator, percent transform, and moment the scorer produces: each continuous metric `m` yields a robust (trimmed) mean kept as `m` plus its median `med_m`, and each logical flag `f` yields the share (percent) of the group flagged, `pct_f`. A few producers with near-zero baseline revenue produce extreme indices, so the means are trimmed (see `trim_mean`) and the medians reported alongside as a robustness check. Every summary accepts an optional `weight` column, so results can be expressed per acre, per dollar of liability, or per simulation weight rather than per producer.

Usage

```
summarise_scores(
  dt,
  by,
  weight = NULL,
  lo = 0.005,
  hi = 0.995,
  metrics = NULL,
  flags = NULL,
  medians = TRUE,
  include_moments = TRUE
)
```

Arguments

`dt`	data.table of per-producer scores (from `compute_efficiency_scores()`).
`by`	character vector of grouping columns, or `NULL` for a single overall summary.
`weight`	optional name of a numeric weight column in `dt`. `NULL` (default) weights every producer equally; when supplied, the trimmed means, medians, and flag shares are all weighted by it.
`lo, hi`	trimming quantile cut points forwarded to `trim_mean`.
`metrics`	optional character vector of continuous columns to summarise. `NULL` (default) uses the scores (and, if `include_moments`, moments) from `efficiency_score_columns` that are present in `dt`.

`flags` optional character vector of logical/0-1 columns to report as `pct_*` shares. `NULL` (default) uses the flags from `efficiency_score_columns` present in `dt`.

`medians` whether to also report a `med_*` median for every metric (default `TRUE`).

`include_moments` whether to include the baseline/safety-net `*_base`/`*_sn` moments among the default metrics (default `TRUE`).

Details

Which columns are summarised is discovered from `efficiency_score_columns` intersected with the columns actually present, so non-score columns (identifiers, group codes, the weight) are ignored automatically. Pass `metrics`/`flags` to override.

Value

data.table, one row per `by` group, with `n_producers`, a trimmed mean for every metric, a `med_*` median for every metric (when `medians`), and a `pct_*` share for every flag.

See Also

`compute_efficiency_scores`, `efficiency_score_columns`, `trim_mean`

`toy_outcomes` *Toy outcomes dataset for examples and CI*

Description

A small, committed dataset so examples and the test suite run with no network access. Each insurance pool's calibrated yields *across years* serve as its per-agent outcome draws; the indemnity and premium are illustrative (two regimes, `baseline` and `reform`) and carry no policy meaning.

Usage

```
toy_outcomes
```

Format

A `data.table` with one row per pool x regime x year and columns:

group_id insurance-pool identifier (the agent).

regime `"baseline"` or `"reform"`.

year commodity year (the draw index).

outcome calibrated yield (the end-of-season outcome).

indemnity illustrative shortfall payment below the guarantee.

premium illustrative producer-paid premium.

Source

Built by `data-raw/scripts/build_toy_dataset.R` from the USFarmSafetyNetLab `calibrated_yield` release.

trim_mean *Trimmed (optionally weighted) mean*

Description

Mean of `x` after discarding the tails below the `lo` and above the `hi` quantiles. This guards headline score summaries against the handful of producers whose near-zero baseline revenue makes their indices explode and dominate a plain mean (the same trimming used in Tsiboe et al. 2025). Non-finite values are dropped before trimming. The tails are cut at the (unweighted) `lo`/`hi` sample quantiles of `x` – the extreme values are extreme regardless of weight – and, when `weights` is supplied, the retained values are then averaged with those weights. Equal weights therefore reproduce the unweighted trimmed mean exactly.

Usage

```
trim_mean(x, lo = 0.005, hi = 0.995, weights = NULL)
```

Arguments

`x`	numeric vector to summarise.
`lo`, `hi`	lower/upper quantile cut points (defaults 0.005 / 0.995).
`weights`	optional numeric vector of non-negative weights, the same length as `x`. `NULL` (default) gives the unweighted trimmed mean.

Value

length-one numeric; `NA_real_` when no finite values remain.

Examples

```
trim_mean(c(1, 2, 3, 4, 1000))
trim_mean(c(1, 2, 3, 4, 1000), weights = c(1, 1, 1, 1, 5))
```

variability_reduction_score *Variability Reduction Score (VRS)*

Description

Ratio of an outcome-variability measure with the safety net to the measure without it. `VRS < 1` indicates the configuration reduces variability. Works for ANY variability input (CV, downside semivariance, loss-conditional severity, ...), giving the headline and the robustness flavors.

Usage

```
variability_reduction_score(treated_var, baseline_var)
```

Arguments

`treated_var`, `baseline_var`	numeric vectors of the variability measure with and without the safety net.

Value

numeric vector (non-finite ratios returned as NA).

Index